

خطة الاستكمال المرحلية لمشروع EngineeringOS

الهدف العام

الوصول بالمشروع من **operational skeleton** قوى إلى منصة هندسية متماسكة كاملة، عبر مراحل متتابعة تقلل المخاطر وتربط الطبقات ببعضها: البيانات، التحليل، التنفيذ، الحوكمة، العرض.

مبدأ التنفيذ

لا يتم البدء من الواجهة.
الأولية تكون دائمًا بالترتيب التالي: 1. سلامة البيانات والقيود 2. صلاية التنفيذ والخدمات الخلفية 3. تعميق المعرفة والـ **graph** 4. توسيع الاختبارات 5. تحسين العرض 6. التوثيق والتسليم النهائي

المرحلة 0: تثبيت الحقيقة المعمارية

الهدف

تجميد فهم المشروع الحالي في وثيقة مرجعية تنفيذية واحدة قبل أي تغيير كبير.

الأعمال

- اعتماد سجل الحقيقة الملقي كمرجع رسمي.
- توثيق الطبقات الرئيسية وعلاقاتها.
- حصر المسارات الحرجة: **API, DB, scanner, discovery, tasks, workflows, graph, audit**.
- تحديد الملفات المولدة والملفات اليدوية والملفات التجريبية.

المخرجات

- خريطة طبقات نهائية.
- قائمة اعتماديات بين الجُزم.
- قائمة الملفات الحرجة حسب الأثر.

معياري الإكمال

- لا توجد طبقة غير موضحة.
- لا توجد علاقة رئيسية غير مذكورة.
- لا يوجد التباس بين **what exists** و **what is only aspirational**.

المرحلة 1: تقوية البيانات والقيود المرجعية

الهدف

رفع سلامة البيانات قبل توسيع السلوك التشغيلي.

الأعمال

- مراجعة جميع الجداول ذات العلاقات المنطقية الرخوة.
- تقوية القيود المرجعية حيثما أمكن.
- مراجعة ال nullable fields التي تحمل معنى تشغيليًا حساسًا.
- إضافة invariants على حقول الحالة وال lifecycle.
- مراجعة dedupe / uniqueness في الكيانات والعلاقات.

الأولويات داخل المرحلة

1. events.taskId و events.workflowId
2. scan_jobs.projectId
3. discovery_sessions.importedProjectId
4. أي حقول حالة أو referential links تعتمد على التطبيق فقط

المخرجات

- Schema أكثر صلابة.
- أخطاء أقل في الترابط.
- تقليل احتمال data drift.

معياري الإكمال

- لا توجد روابط منطقية حرجة بلا حماية واضحة.
- حالات البيانات الأساسية أصبحت قابلة للتدقيق بسهولة.

المرحلة 2: تقوية طبقة التنفيذ الخلفية

الهدف

جعل عمليات المنصة الأساسية موثوقة، خلفية التنفيذ فيها واضحة، وأثر الفشل قابلاً للاحتواء.

الأعمال

- مراجعة scan-runner لتقليل أي fallback غير مقصود.
- مراجعة tasks لتقليل partial writes.

- مراجعة workflows لتحديد نقاط التوقف والانتقال بوضوح.
- جعل الأخطاء الحرجة مرئية في السجل وفي الأحداث.
- توحيد حالات التشغيل بين المشروع والـ jobs والمهام.

الأولويات داخل المرحلة

1. scan jobs
2. discovery import
3. task execution/retry/rollback
4. workflow start/stop/executions

المخرجات

- Execution layer أكثر ثباتًا.
- سيناريوهات الفشل أقل تشعبًا.
- trace أوضح لكل عملية.

معياري الإكمال

- كل عملية تشغيلية رئيسية لها lifecycle مفهوم.
- أي فشل لا يترك النظام في حالة غامضة أو نصف مكتملة.

المرحلة 3: تعميق scanner وتحويله إلى intelligence layer أقوى

الهدف

رفع قدرة المنصة على فهم المستودع من مسح سطحي إلى تحليل أعمق وأكثر دلالة.

الأعمال

- توسيع تغطية graph-extractor خارج TS/JS.
- تحسين دعم CommonJS و require و export =.
- مراجعة heuristic parsing في Python.
- تحسين rule matching ليصبح أكثر دلالية وأقل اعتمادًا على regex فقط.
- ربط نتائج scanner مباشرة بـ graph و metrics و tasks.

المخرجات

- اكتشاف أكثر ثراء للعلاقات.
- نتائج scan أدق وأقرب إلى الحقيقة البنيوية.
- مدخل أقوى لتغذية باقي الطبقات.

معييار الإكمال

- scanner يلتقط العلاقات الجوهرية بدرجة مقبولة في أهم اللغات المستهدفة.
- نتائج scan تصبح قابلة للاعتماد في القرار التشغيلي، لا مجرد تقرير.

المرحلة 4: تحويل graph إلى طبقة معرفة تشغيلية

الهدف

الانتقال من graph كبيانات إلى graph كطبقة فهم واستكشاف وقرار.

الأعمال

- مراجعة `graph_entities` و `graph_relationships`.
- إزالة التكرار غير الضروري في العلاقات.
- إضافة semantics أوضح للعلاقات.
- ربط graph بنتائج scanner و scan jobs و discovery.
- تحسين العرض البصري بحيث يعكس العلاقات الحقيقية لا التوزيع الاصطناعي.

المخرجات

- graph يمكن استكشافه كمعرفة حقيقية.
- فهم أفضل للعلاقات بين الملفات والوظائف والمهام.
- طبقة تفسير أقوى للمشروع.

معييار الإكمال

- يمكن تتبع علاقة الملف بالوظيفة بال task بال workflow بشكل أوضح.
- graph لم يعد مجرد visualization، بل مرجعًا بنيويًا.

المرحلة 5: تحويل workflows إلى engine فعلي

الهدف

جعل workflows أكثر من كيان وصفي، وتحويلها إلى orchestration engine قابلة للتنفيذ المرحلي.

الأعمال

- تعريف states والانتقالات بين المراحل.
- دعم branching أو decision points عند الحاجة.

- إضافة retry على مستوى المرحلة.
- إضافة rollback على مستوى المرحلة أو على مستوى workflow execution.
- ربط workflow execution بالـ tasks/events/audit.

المخرجات

- workflow engine أوضح.
- تنفيذ متعدد المراحل يمكن تتبعه واختباره.
- ارتباط أقوى بين التصميم والتنفيذ.

معياري الإكمال

- كل workflow execution يمكن تفسيره مرحلة بمرحلة.
- الفشل في مرحلة لا ينسف المنظومة كلها بدون أثر واضح.

المرحلة 6: توحيد التتبع والحوكمة

الهدف

بناء traceability موحدة بين الأحداث، المهام، التدقيق، والقياسات.

الأعمال

- ربط `audit_logs` و `events` و `task_logs` و `metrics` بسجل سردي واحد.
- تقوية `recordAudit` في المسارات الحرجة.
- توحيد naming و correlation IDs حيثما أمكن.
- جعل الأحداث أكثر قابلية للتصفية والتحليل.

المخرجات

- trace موحد لكل عملية.
- حوكمة أوضح.
- سهولة في التحقيق والتشخيص.

معياري الإكمال

- يمكن تتبع أي عملية من البداية إلى الأثر النهائي.
- لا توجد فجوات كبيرة بين ما حدث وما تم تسجيله.

المرحلة 7: توسيع الاختبارات

الهدف

تحويل الثبات من اعتماد على البنية إلى اعتماد على التحقق.

الأعمال

- توسيع الاختبارات خارج scanner و projects/discovery.
- إضافة coverage ل tasks.
- إضافة coverage ل workflows.
- إضافة coverage ل graph.
- إضافة coverage ل metrics و plugins.
- إضافة اختبارات تكامل للمسارات الحرجة.

المخرجات

- تقليل regressions.
- ثقة أعلى في التعديلات القادمة.
- قدرة أفضل على تطوير طبقات معقدة بدون كسر المنظومة.

معياري الإكمال

- لا تبقى الطبقات الأساسية خارج التغطية.
- المسارات الحرجة محمية باختبارات وحدات وتكامل.

المرحلة 8: تحسين الواجهة لتعرض الحقيقة الداخلية

الهدف

جعل الواجهة انعكاسًا دقيقًا لعمق المنصة، لا مجرد طبقة عرض مبسطة.

الأعمال

- تعميق صفحة Graph.
- تحسين Workflows لتعرض execution states و phases.
- توضيح أثر tasks و discovery و scan على المشروع.
- تقليل الاعتماد على polling حيثما أمكن.
- تحسين التمثيل البصري للمقاييس والحوكمة.

المخرجات

- تجربة تشغيل أوضح.
- فهم أسرع لحقيقة المشروع الداخلية.
- تقليل الفجوة بين الداخل والخارج.

معياري الإكمال

- المستخدم يرى ما يحدث فعليًا داخل النظام، لا نسخة مختزلة منه.

المرحلة 9: التوثيق النهائي والتسليم

الهدف

إخراج نسخة رسمية قابلة للتمرير إلى وكيل التطوير أو إلى فريق بشري.

الأعمال

- تثبيت سجل الحقيقة الملقى.
- تحويل الخطة إلى backlog تنفيذي.
- إعداد runbook للتشغيل والصيانة.
- إعداد prompt تنفيذ نهائي لوكيل Replit أو أي agent مشابه.
- توثيق المخاطر المفتوحة والديون التقنية المتبقية.

المخرجات

- وثيقة مرجعية نهائية.
- قائمة تنفيذ مرتبة بالأولوية.
- دليل تشغيل وصيانة.

معياري الإكمال

- يمكن لأي فريق جديد فهم الوضع الحالي وخطة الإكمال من الوثائق دون الحاجة لإعادة الهندسة من الصفر.

التسلسل العملي المقترح للتنفيذ

المسار الأسرع والأكثر أمانًا

1. تثبيت الحقيقة المعمارية.
2. تقوية البيانات والقيود.
3. تقوية التنفيذ الخلفي.

4. تعميق scanner.
5. تحويل graph إلى معرفة تشغيلية.
6. تحويل workflows إلى engine.
7. توحيد الحوكمة والتتبع.
8. توسيع الاختبارات.
9. تحسين الواجهة.
10. التسليم النهائي.

المخاطر إن بدأنا من المرحلة الخطأ

- البدء من الواجهة قبل تثبيت البيانات سيخلق عرضًا جميلًا لحقيقة غير ثابتة.
- البدء من workflow قبل data integrity سيزيد من صعوبة الإصلاح.
- توسيع scanner قبل ربطه بال graph و metrics سيجعل النتائج معزولة.
- إهمال الاختبارات سيحوّل أي تقدم إلى هشاشة مستقبلية.

الخلاصة التنفيذية

هذه الخطة تُبنى على حقيقة أن EngineeringOS:

- ليس مشروعًا بسيطًا.
- ليس مجرد dashboard.
- ليس مجرد scanner.
- بل نظام متعدد الطبقات يحتاج تدرجًا في الإكمال.

أفضل مسار هو: تثبيت → تشغيل → فهم → ربط → اختبار → عرض → تسليم.

وبهذا الترتيب فقط يصبح المشروع جاهزًا للنضج الحقيقي.